Introduction:

The aim of this project is to build a supervised machine learning model to classify credit card customers in default and non-default based on their payment and billing history. A credit card default arises when a user fails to make payments for an extended duration.

The central objective involves designing an autonomous system capable of both isolating critical determinants and forecasting the likelihood of default, grounded on a customer's profile and behavioural history. The broader principles of supervised machine learning are outlined, followed by a technical walkthrough of the chosen algorithms. Specifically, the models employed include Logistic Regression, Decision Trees and ensemble techniques like XGBoost and LightGBM.

Development Stack

The whole analytical code was entirely implemented using the Python programming language, leveraging libraries like scikit-learn, NumPy, pandas, and imbalanced-learn, along with visualization packages such as matplotlib and seaborn, to perform both modelling and exploratory data examination. Others libraries are mentioned in code.

EDA:

The dataset contains credit card client information with 25,247 entries and 26 original features, expanded to 32 after one-hot encoding categorical variables.

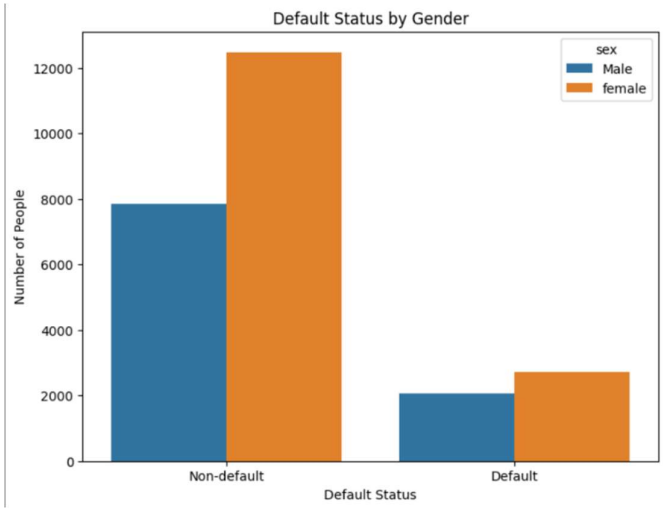
Information delivered by different columns:

1. Customer_ID: This column doesn't have any significance in predicting default so this has been dropped.
2. PAY_0 represents the payment status in the most recent month (Month 0), PAY_2 represents the payment status one month before that, and so on. Values: -2 = No credit consumption (no bill) in month m−1. -1 = Bill generated and fully paid in month m−1 (same month payment) 0= Partial or minimum payment made (revolving credit). ≥1 = Payment delayed by that many months (1 = 1 month overdue, etc.)
3. Marriage: Marital status of the customer (1 = Married, 2 = Single, 3 = Others)
   This column was having more than these three values, them are treated as others (i.e. = 3)
4. Sex: Gender of the customer (1 = Male, 0 = Female)
   These are ditribution of male and female accros defaulters and non defaulters.
5. education: Education level (1 = Graduate School, 2 = University, 3 = High School, 4 = Others)
   This column also has some values other than these 4 mentioned that are treated as others.

```
marriage
2    13374
1    11424
3      270
0       53
Name: count, dtype: int64
marriage
2    13374
1    11424
3      323
Name: count, dtype: int64
```

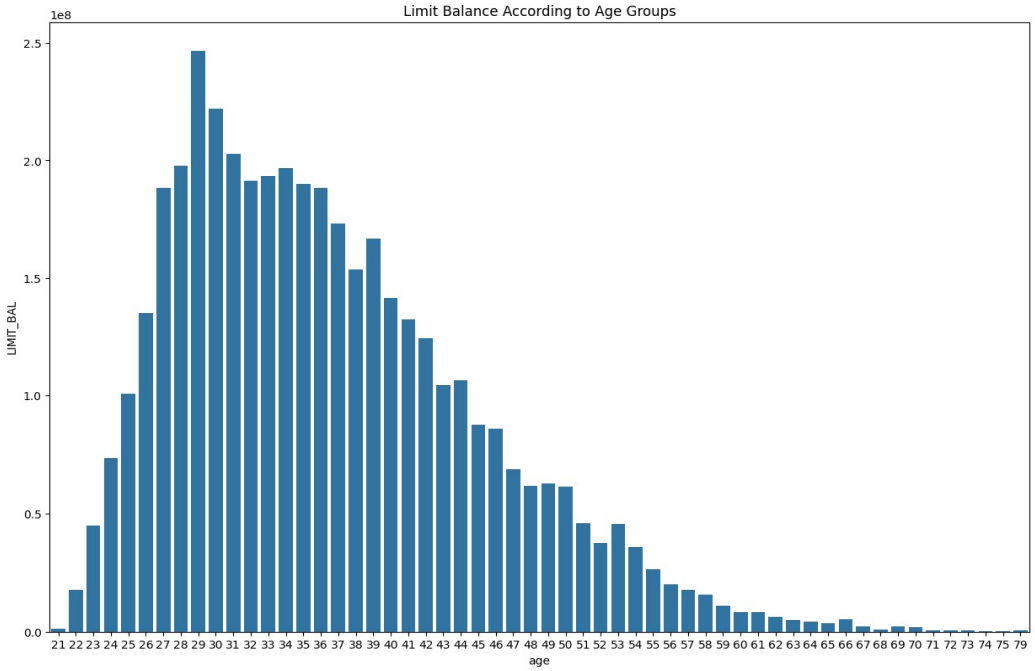
```
sex
1    15191
0     9930
Name: count, dtype: int64
```

```
education
2    11657
1     8944
3     4096
5      252
4      115
6       43
0       14
Name: count, dtype: int64
education
2    11657
1     8944
3     4096
4      410
0       14
Name: count, dtype: int64
```



6. LIMIT_BAL: Average credit limit: 168,342 (ranging 10k - 1M). Credit limit assigned to the customer (in currency units)
7. age: Mean age- 35.4 years (21 - 79 range)

These figure shows distribution of limit balance across different age. The aggregation of limit



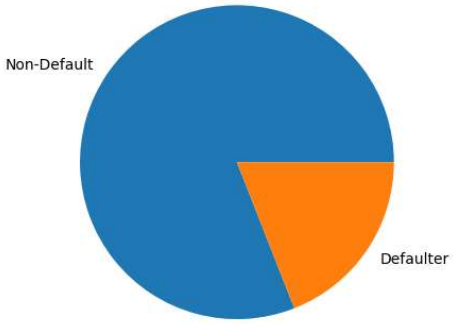
balance increases from 21 to 29 and decreases further. Younger people have given more limit balance as their score might be good due to recent interaction with credit card. While the limit balance decreases with further age as older people might not use credit card frequently and their score might get bad as they have been more interacting with credit cards.

Also, this column has 126 null entries which is been dropped as our data is big which doesn't affect much.

8. Bill_amt_m(Bill_amt1 to 6): Total bill amount at the end of month *m* (1 = last month, 6 = six months ago). > 0 means customer owes money (paid less than previous bill). = 0 no spending. < 0 customer overpaid previous bill (credit carried)
9. Pay_amt_m(Pay_amt1 to 6): Payment amount made in month m towards the bill generated in month m-1.

Duplicates was seen in the dataset and has been remove using drop_duplicate()

Target variable: **next_month_default**: 1 if customer defaulted next month, 0 otherwise.




```
next_month_default
0    20337
1     4784
Name: count, dtype: int64
```

Our data nearly contain 19% as possitive cases(default) and 81% as negative cases(non-default).
These number shows that our data is significatly imbalance and some efficient techniques must be considered to properly address this or model will be biased.

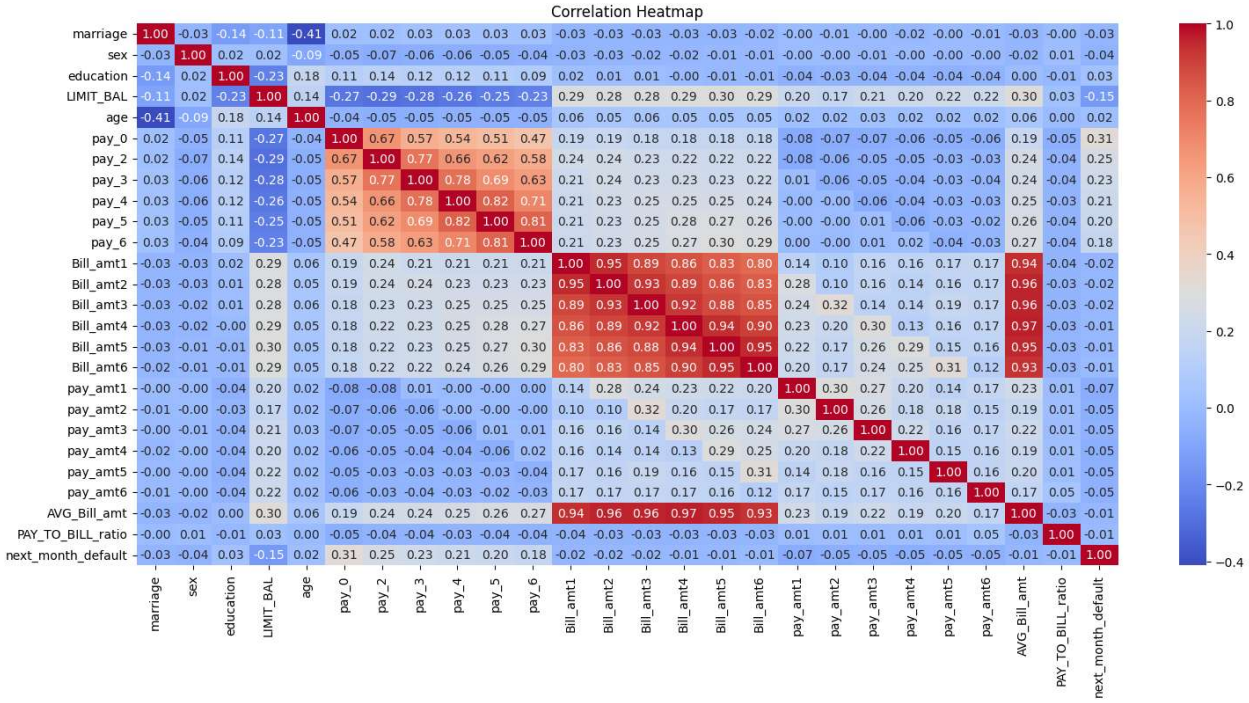
Addressing categorical variables-

Categorical variable such as sex, marriage, education – they have been One-hot encoded as numerical encoding might impose some oridinal relationship which has no meaning.

Correlation between features

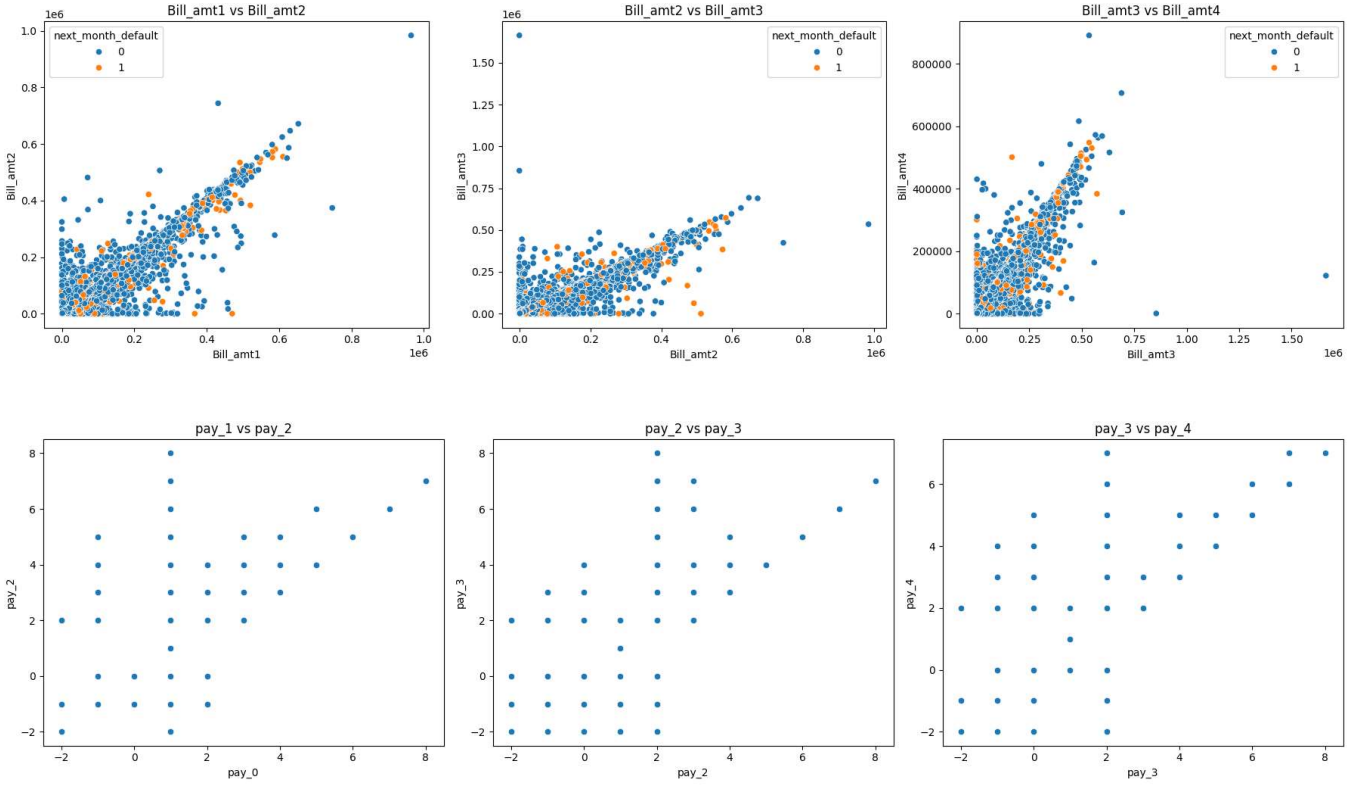
When building classification models, it's important to look at how features relate to one another. If some features are strongly correlated, it can negatively impact the performance of certain algorithms particularly those that assume feature independence. Identifying these correlations can also be useful for simplifying the model, as highly similar features may be redundant and offer no additional value.

Multicollinearity affects the performance of linear classification model. As many features having redundant information affects the classification of models.



Above given heatmap represent correlation between any two features. As we can see Bill_amt1-Bill_amt6 has very strong correlation in between them. And another set of highly correlated features are pay0-pay6.

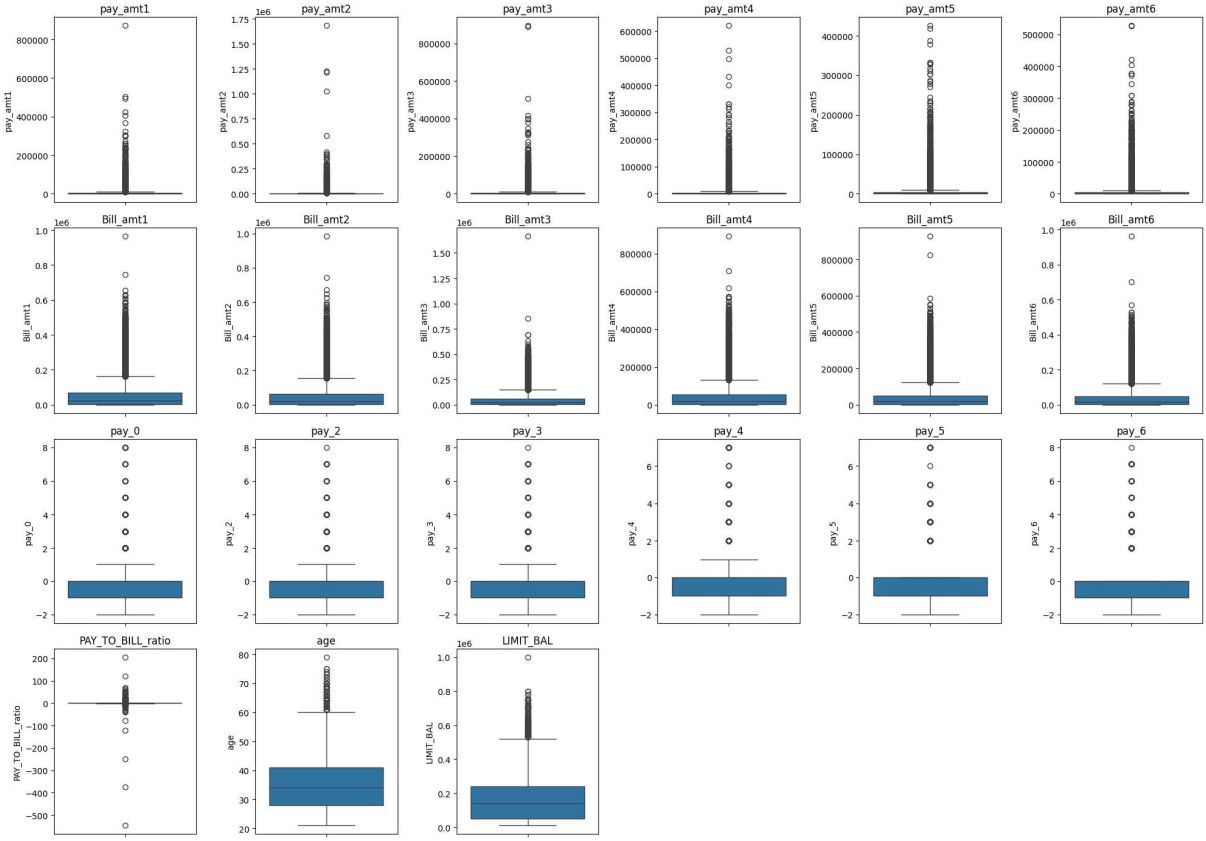
Further to visualise this collinearity we plot scatter plots among the collinear feature to visualise the trend.



The plots confirm what's expected the feature in the same graph shows a linear trend.

To address this collinear feature certain measure are taken which will be explained further.

Outliers:



Above are the box plots of numerical features present in data. There are several outliers in present in each of them. And due to significant contribution of them in predicting default none of them are dropped. As dropping or changing them will results in significant loss of data.

Feature Engineering:

Here I have opted two methods for Feature Engineering:

1. Constructed features based on domain knowledge.
2. PCA (Principal component analysis).

Constructed features based on domain knowledge:

I have constructed some features to better explain the trend in default and non-default.

1. **Pay to bill ratio:**

   Pay_to_bill = (total payment made/ total bill generated)

Understanding Pay_to_Bill Ratio. 0 = No payment made. 0.5 = Paid 50% of the bill. 1.0 = paid the full bill amount. > 1.0 = Paid more than the bill (overpayment).

The histogram is plotted with bin size 0.1 from min = 0 and max = 5. The data is observed to be highly right skewed.

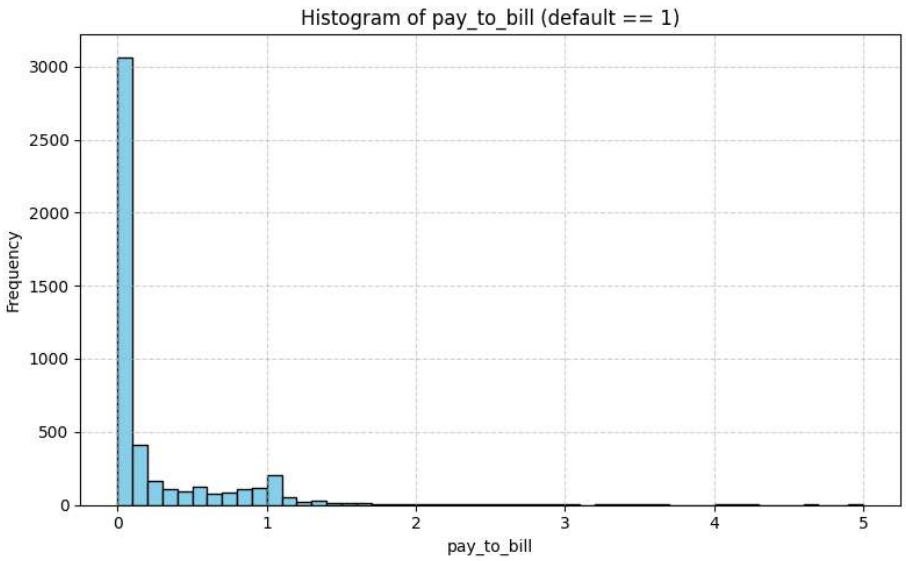
Key patterns in data:

Dominant Non-Payment Behaviour: The big spike at 0-0.1 (~3,000) shows that most defaulters made no payments at all relative to their bills. This represents the most common pattern among defaulters.

Partial Payment Group: There are some number of defaulters paying roughly 30 – 70% of their bill amounts. This suggests a subset of defaulters who made partial payments but still couldn't avoid default.

Minimal Full Payment: Very few defaulters show pay_to_bill ratios at or above 1.0, indicating that customers who pay their full bills rarely but still end up defaulting.

Right-Skewed Distribution: The distribution is heavily concentrated in the lower ratios (0-0.5), which aligns with expectations since these are customers who ultimately defaulted.



The extremely low frequency of higher pay_to_bill ratios among defaulters reinforces that consistent, adequate payment behaviour is strongly associated with avoiding default.

2. **Utilization:**

Utilization = (avg bill generated/limit balance)

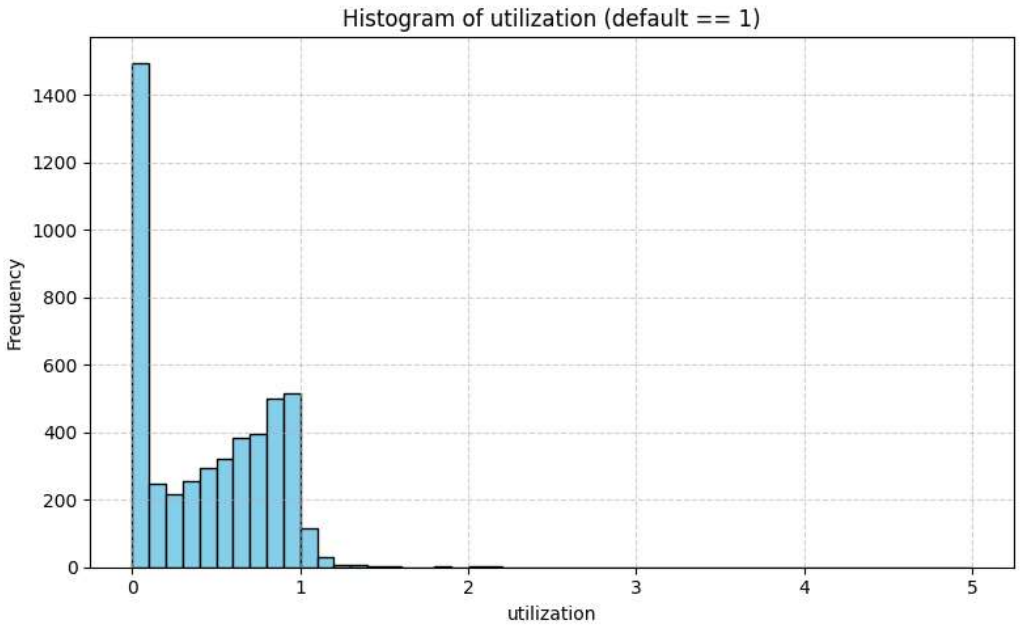
The percentage of available credit a borrower is using—plays a central role in predicting and explaining why some individuals become credit card defaulters.

Why Credit Card Utilization Matters?

1. **High Utilization Signals Financial Stress:** Borrowers who use a large portion of their available credit (often defined as 50-90% or more) are much more likely to face financial problem. This high utilization often indicates that individuals don't have other source of income and using credit cards for its daily expenditure. which can lead to delays in making payments.
2. **Strong Predictor of Default:** Studies shows a direct correlation between high credit card utilization and future delay in repayments.
3. **Impact on Credit Score:** Utilization is one of the most important factors in understanding default pattern. High utilization lowers credit scores, making it harder to access affordable credit in the future and hence high interest rates.

   Visualising utilization in our dataset:

   I have plotted a histogram of defaulter data with bin size = 0.1.



The data shows a **2**:**1 ratio** of defaulters between high (>0.3) and low (<0.1) credit utilization groups. This can be explained through several behavioural factors:

1. **High Utilization Group** (>**0.3**): Borrowers crossing the 0.3 threshold often face compounding interest (typically 18-48%) that makes debt repayment increasingly difficult. These users likely rely on credit card for essential expenses, leaving minimal margin for unexpected costs. In our data there are nearly ~2800 entries are such which explain this trend.
2. **Low Utilization Group** (<**0.1**): Customers of this group may be first-time borrowers with limited credit history who underestimate repayment obligations. Some of them may maintain low card utilization while carrying high debt through other sources

(personal loans, etc). Sudden job loss or medical emergencies can affect this group's repayment capacity. These reasons perfectly explain this group.

Why sudden rise at 0 – 0.1?

It can be explained through financial distress - Some borrowers stop using cards entirely after financial distress begins, creating artificial low utilization before default. Some of them might be using credit card for first time may not have taken repayment seriously and also low awareness towards they credit score.

Above insights suggest some measures to be followed:

1. Monitor even low-utilization accounts for sudden payment behaviour changes.
2. Develop early-warning systems combining utilization trends with cash flow analysis.
3. Offer targeted financial literacy programs for new credit users.

3. Weighted Average Delay:

In this feature I have calculated average delay by assigning higher weights to recent payment behaviour (pay_0 = 6, pay_1 = 5, etc.) as it makes strong business sense because recent payment patterns are more predictive of future default risk than older payment history.

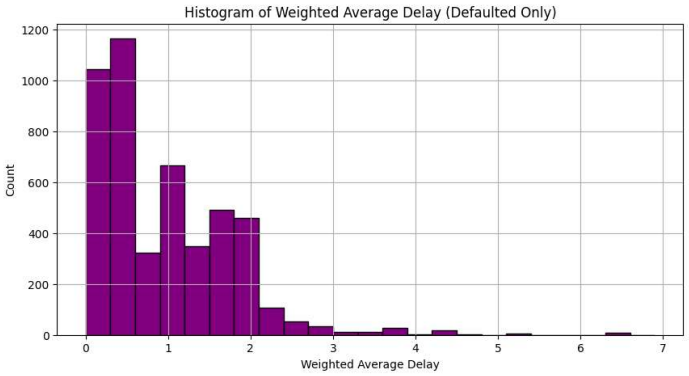
Method:

Here I have categorised 3 payment behaviour:

a. Normal payment behaviour: In this section the values with -1 and -2 falls. Which respectively indicate fully paid and no bill generated. I have converted them to 0.
b. Minimum payment behaviour: This section holds those who made minimum payments consistently. These indicate a 0 in our original column, so I changed it to 0.5 to reflect some risk factor.
c. Delayed payment behaviour: Preserving the (1-8) month delay scale maintains the severity gradient.

WAD = (6*pay_0 + 5*pay_2 + 4*pay_3 + 3*pay_4 + 2*pay_5 + pay_6)/ (21)

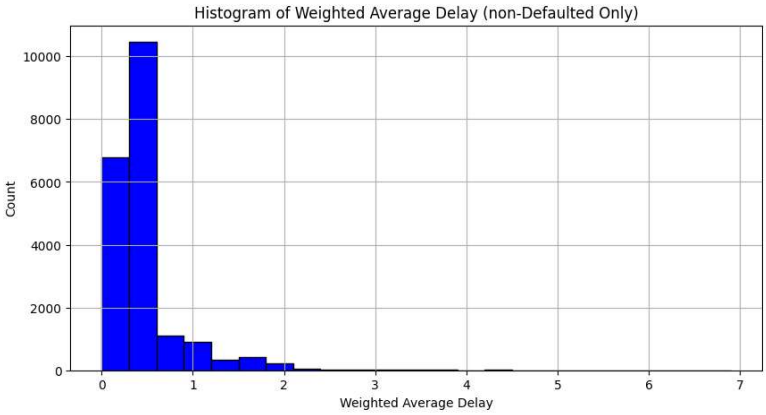
The first histogram is plotted with bin size = 0.3 and taking only defaulter. The later one is of non-defaulter.



Interpreting the graphs:

- Most non-defaulters have a weighted average delay between 0 and 0.6.
- There is a sharp drop in counts after 0.6, with very few non-defaulters having higher delay scores.
- Defaulters are spread more broadly across the weighted average delay axis.

- There is still a significant number of defaulters in the 0–0.6 range, but unlike non-defaulters, many defaulters have higher delay scores, with a long tail extending beyond 2.0 and even up to 7.0.



Most non-defaulters are in range (0-0.6), showing consistently timely or near-timely payments. Many defaulters also appear here, indicating that not all defaults are preceded by long or frequent delays—some defaulters may have had sudden financial shocks, or their risk is not captured solely by payment timing. Above 0.6, non-defaulters drop off sharply, indicating that sustained or severe delays are rare among those who do not default. Defaulters remain present, showing that higher weighted delays are strongly associated with default risk.

Why customers less than 0.6 WAD are defaulted?

- **Sudden Default**: Some customers default abruptly due to unexpected things happening in life (job loss, illness, etc) without a long history of payment problems.
- **Other Risk Factors**: Default can be driven by factors outside payment history such as high utilization, external financial stress or sudden losing their source of income.
- **Minimum Payments**: Customers making only minimum payments (scored as 0.5) may appear low risk by this metric but could still be financially strained.

Key insights from this observation:

- The weighted average delay feature is highly effective for distinguishing risk groups, especially above the 0.6 threshold.
- The overlap below 0.6 highlights that default is multifactorial; payment history is necessary but not sufficient for perfect prediction.
- This feature should be used alongside other variables (utilization, limit balance, other data) for good credit risk modelling.

PCA (Principal component analysis):

One effective approach to reduce dimensionality—particularly in the presence of multicollinearity is **Principal Component Analysis (PCA)**. This is especially useful when working with unfamiliar datasets and limited domain expertise. PCA is an unsupervised learning technique that transforms the original features into a new set of uncorrelated components, called *principal components*, using a linear, orthogonal transformation.

These components are ranked based on how much variance they explain in the data. The key idea is that while original features may not align with directions of highest variability, PCA

identifies those directions and transforms the data along them, retaining the mostly data which explains the original data.

Feature scaling: *Standardization* is applied to all the columns having numerical data.

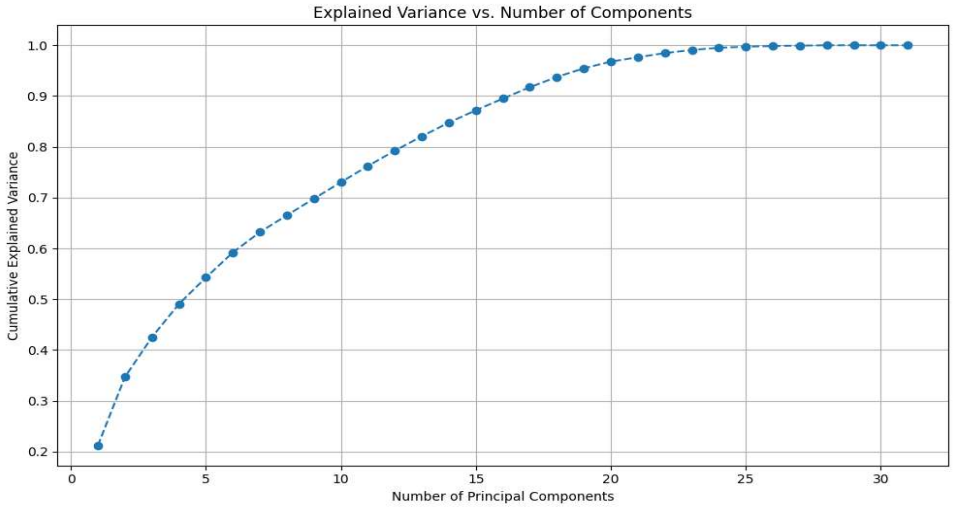
Standardization is a feature scaling technique that transforms data to have a mean of 0 and a standard deviation of 1. It is done using the formula:

$$z = x - \mu / \sigma$$

where x is the original value, μ is the mean, and σ is the standard deviation.

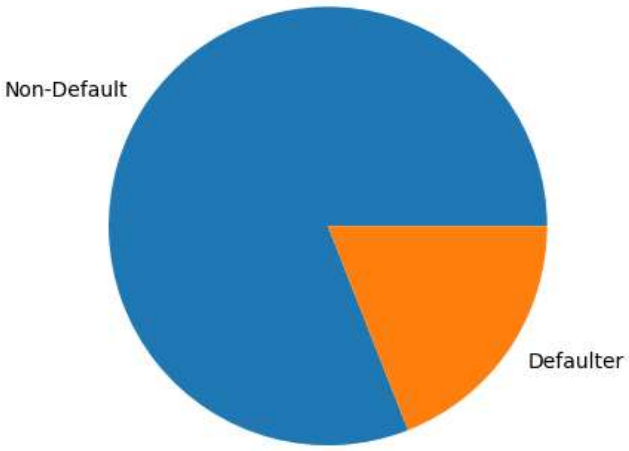
Note that statistics for scaling is only computed on training data (μ and σ). And then both data set are transformed according to them. As it prevents leaking any information from test set to training set.

```
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
X_train = pd.DataFrame(X_train, columns=X.columns)
X_test = pd.DataFrame(X_test, columns=X.columns)
```



Number of Principal Components taken = 20 as 95% of variance is explained by cumulative of first 20th Principals.

Handling class imbalance:



Total 19% of positive cases are present with 81% of negative cases. This shows our data is imbalanced and need to be addressed.

To address this issue there are several sampling techniques. Some of them are:

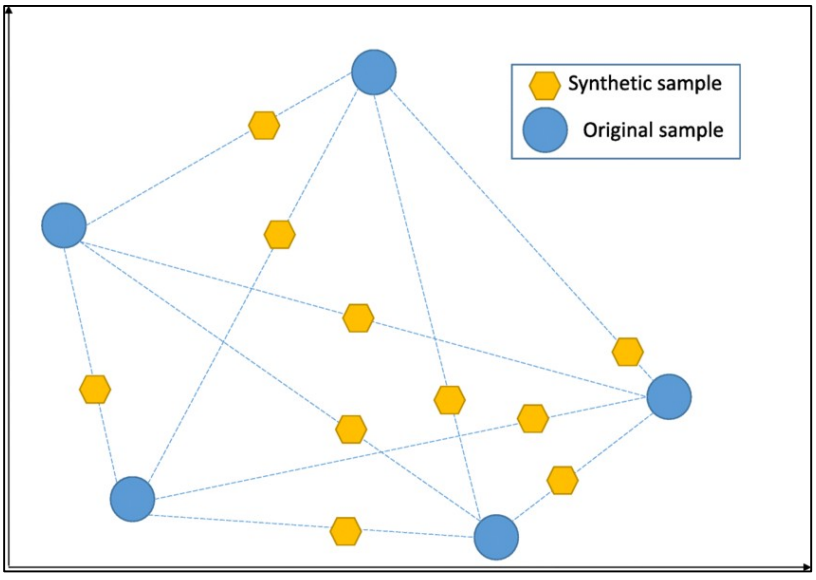
- Under sampling.
- Random Oversampling.
- SMOTE.

Here we are going to use SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE:

SMOTE (Synthetic Minority Oversampling Technique) is a method to balance the imbalanced data sets by creating new, synthetic samples from the minority class rather than simply duplicating existing ones. It works by picking a minority-class point, finding its *k* nearest neighbours (usually k = 5), then generating synthetic points along the line segments that connect the original point to randomly chosen neighbours.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_sampled , y_train_sampled = smote.fit_resample(X_pca_train_val, y_train)
```



Train test split:

```
target_col = 'next_month_default'

X = data.drop(columns=[target_col])
y = data[target_col]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

Here data set is split into train and test with ratio of **3:1.** This split is done before applying any scaling and sampling technique. As it avoids leaking information from test to train.

Evaluation Metrics:

There are 5 major types of Metrics upon which we evaluate our model:

- Recall
- Precision
- F1 score
- Accuracy
- AUC-ROC score

Accuracy:

Only evaluating accuracy can be misleading here. Defaults are rare (e.g. 15–19% of accounts). A dumb "always-good" model would give more accuracy than any other properly trained model. It hides the fact that we are missing most of the Defaulters.

Recall:

Why giving priority to recall is important as it correctly distinguishes false negative. It has real world implications. Having High recall suggests we are minimizing losses by identifying most of the Defaulters. This should be given high priority.

Recall = TP / (TP + FN)

These all metrics are worth noting but as minimizing the losses is more important than alarming a false one. Minimizing False Negatives is our priority.

This is done by F2 score.

$$F_\beta = \frac{\beta^2 + 1}{(\beta^2 \cdot \text{recall}^{-1}) + \text{precision}^{-1}} = \frac{(1 + \beta^2) \cdot \text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}}$$

The F2 score is a way to measure a model's performance when identifying positives (defaulters) is more important than being exactly right. It focuses more on recall, so it's better for situations where missing a defaulter is worse than wrongly predicting one.

Training model and Results:

As mentioned earlier I have opted two different methods to reduce dimensionality one by using PCA and one by applying domain knowledge.

Results and metrics score after applying PCA:

Top 22 PCA components are taken to train the models, with standard scaling used. Optimal threshold has been found by looping over threshold and comparing parameter was kept being F2 score. Peak F2 threshold was given below. Hyperparameter was also tuned.

Model	*Best Threshold*	*Accuracy*	*Precision*	*Recall*	*F1*	*F2*	*ROC AUC*
DecisionTree	0.19	0.4648	0.2440	0.8202	0.3762	0.5571	0.6993
XGBoost	0.34	0.5390	0.2780	0.8411	0.4179	0.5986	0.7594
LightGBM	0.31	0.4921	0.2612	0.8654	0.4013	0.5917	0.7607
Logistic	0.35	0.4339	0.2388	0.8587	0.3737	0.5653	0.7234
AdaBoost	0.49	0.3901	0.2350	0.9314	0.3753	0.5848	0.7526
Support Vector Machine	0.30	0.4980	0.2649	0.8746	0.4067	0.5989	0.7554

SVM is observed to be best among all with highest F2 score of **0.60** with accuracy of **0.50**.

Results and metrics after Feature Engineering:

Total 22 feature were used to train the model. With some of newly constructed features and some highly correlated features were dropped to reduce multicollinearity. (pay_3 – pay_6) and (Bill_amt3 – Bill_amt6).

New feature constructed – Utilization, Weighted Average Delay, Pay_to_Bill_ratio.

pay_0, pay_2, Bill_amt1 and Bill_amt2 were kept as recent payment behaviour and Bill amount explains Default trend.

Optimal threshold has been found by looping over threshold and comparing parameter was kept being F2 score. Peak F2 threshold was given below. Hyperparameter was also tuned.

Model	*Best Threshold*	*Accuracy*	*Precision*	*Recall*	*F1*	*F2*	*ROC AUC*
DecisionTree	0.11	0.5590	0.2737	0.7508	0.4012	0.5567	0.6961
XGBoost	0.18	0.5648	0.2884	0.8261	0.4275	0.6017	0.7628
LightGBM	0.20	0.5839	0.2979	0.8219	0.4373	0.6080	0.7762
Logistic	0.39	0.3906	0.2311	0.9013	0.3679	0.5704	0.6579
AdaBoost	0.49	0.7025	0.3643	0.6881	0.4764	0.5843	0.7485
Support Vector Machine	0.34	0.3590	0.2250	0.9239	0.3619	0.5699	0.6433

Here **LightGBM** is observed to be best among all models with highest F2 score of **0.61** and accuracy of **0.58**.

Choosing model in view of different bank strategies:

1. Revenue Generating Banks:
    a. *Lower False Positive Rates:* Banks who want to maximise their customer acquisition and ready to accept some of the default risk.
    b. *Revenue Preservation:* As fewer good customer will be flagged as default. Which will improve banks relationship with customers, and it will maintain transaction flow from profitable customers. This retains more profitable customers while tolerating little more Default.

    These banks must focus on Accuracy as well as F2 score, a small trade-off in F2 score is tolerable.

    So **AdaBoost** from second approach suit this strategy as it has F2 score of **0.58** and highest accuracy among all of **0.70**.

2. Loss Reduction Banks:
    a. *Maximum Default Detection:* These banks are well established banks while having sufficient customer relationships. So these want to minimize their financial losses from defaults while maintaining operational viability.
    b. *Conservative Growth:* As having good customer base this bank majorly focuses on catching defaulter early as they may contribute significantly to losses.

    These banks must focus completely on F2 score as their priority is to reduce false negative.

Therefore **LightGBM** will be more robust model for reducing false negatives as it has highest F2 score among all of **0.61**.